Choosing the improver: Codex vs OpenCode (executor fixed to Codex)

Date: 2026-11-05, 16:35 IST **Status:** complete, 2 tasks with one pipeline run per improver

1. Question and answer so far

With **Codex as the executor** (chosen from the four-runtime comparison), which runtime should run the **improver**: Codex or OpenCode?

The Codex improver is ahead. It improved one of the two tasks: `format_errors` went from 8 to 11 of 14 (+3), and `stream_release` stayed at 8. **The OpenCode improver improved neither task**: `format_errors` stayed at 2 and `stream_release` stayed at 9.

Task	Codex improver	OpenCode improver
`format_errors` (14)	8 to **11** (+3)	2 to 2 (0)
`stream_release` (12)	8 to 8 (0)	9 to 9 (0)
Total tests gained	**+3**	**0**

These are two tasks with one run each, and base scores varied a lot between identical setups, so this is a lead, not proof.

2. Setup (identical for both arms)

Item	Arm A: Codex improver	Arm B: OpenCode improver
Executor	Codex, `sofie-code-0`	Codex, `sofie-code-0`
Improver model	`sofie-think-0`	`sofie-think-0`
Improver runtime	Codex CLI in a Docker container (local patch)	OpenCode on the host (the method's default)
Harness files edited	`AGENTS.md`, `.agents/skills/`, `scripts/`, `.codex/`	same
Grader	hidden tests	hidden tests
Run label (separate)	`wb-code-cx-agg1-s1`	`wb-code-cx-agg1-s2`

Task choice. A 6-task screen of the Codex base agent found which tasks Codex partly fails. `format_errors` (11/14) and `stream_release` (9/12) were the closest to solved, so they were picked.

3. Results: `testing-medium-format_errors` (14 hidden tests)

Stage	Codex improver	OpenCode improver
Base (before)	8	2
INV candidate	2	2
DOM candidate	9	8
Final (after)	**11**	**2**
Change	**+3**	**0**

What each improver changed in the final harness - **Codex improver:** skill `pinning-behavior-tests` + an `AGENTS.md` edit. The final beat both its candidates. - **OpenCode improver:** an `AGENTS.md` edit, a Codex hook (`.codex/hooks.json`) and a script `markup_test_run_hints.py`. Its DOM candidate reached 8/14, but AGG built a different harness that scored the same as the base.

Caveat. The bases differ by 6 tests (8 against 2) under identical setups. That's run-to-run variance in the agent. So the size of the difference between arms is uncertain, though only arm A improved over its own base.

4. Results: `reliability-hard-stream_release` (12 hidden tests)

Stage	Codex improver	OpenCode improver
Base (before)	8	9
INV candidate	5	9
DOM candidate	9	4
Final (after)	**8**	**9**
Change	0	0

Neither improver helped on this task. Each composed a final that matched its own base: the Codex improver's final missed its better DOM candidate (9), and the OpenCode improver's final matched its INV candidate (9).

5. Improver speed (`sofie-think-0`, per session)

	Codex improver	OpenCode improver
INV sessions	18.5, 20.5 min	8.8, 17.5 min
DOM sessions	10.8, 27.1 min	7.5, 31.8 min
AGG sessions	14.8 min	18.3 min

The speeds are similar. Neither improver is clearly faster.

6. Problems and fixes during this comparison

1. **Job file collision.** Two runs launched with the same label shared a job file in the benchmark, so the first attempts failed to start. Fix: a different seed per run, and the results folder is now part of each job file's name, so parallel runs can't collide.

2. **Hung improver sessions.** Four sessions (two per arm) stalled for 2.5 to 3 hours on `think-0` responses that never finished. Codex and OpenCode had no hard time limit. Fix: a 60-minute hard cap on every improver channel. The runs resumed from cached work.

7. Recommendation

Use the Codex improver with the Codex executor.

- It produced the only gain of either arm (+3 on `format_errors`), and that final beat its own best candidate.
- It ran at the same speed as the OpenCode improver.
- Running the executor and improver in the same runtime keeps the setup simpler: one runtime and one harness layout to maintain.

Caveats: - The Codex improver depends on my local patch (Codex in Docker, through the bridge). The OpenCode improver is the method’s built-in default. - With two tasks and one run each, a single lucky or unlucky run could change this. To firm it up, repeat 2 to 3 runs per arm on these tasks. The API keys expire at **09:50 IST on 2026-11-06**.